Verdict-Optimized Trace Abstraction for Code Behavior Classification: An Empirical Study of Trace Granularity in SFT

Anonymous

Abstract

Execution traces have been explored as auxiliary inputs for code language models, yet prior work focuses on generative tasks and reports limited benefit. We study trace-augmented supervised fine-tuning (SFT) for *verdict tasks*—binary classification of code behavior (e.g., test pass/fail)—and find that trace design granularity, not trace presence, determines effectiveness. We introduce **Verdict-Optimized Trace Abstraction (VOTA)**, a compact trace format that captures only decision-relevant execution state: assertion-point actual/expected value comparisons, exception summaries, and loop compression. On test verdict prediction with CodeLlama-7B, VOTA-SFT achieves 98.45% accuracy (95% CI: [97.5%, 99.3%]; +23.3pp over the no-trace baseline), while full execution trace SFT collapses to 28.4% (−46.7pp). Component ablation reveals that actual/expected value pairs contribute 74.3% of VOTA's lift, identifying the core mechanism as task-aware information selection rather than behavioral reasoning. We further show that full-trace collapse stems from two factors: gradient dilution (∼60%) and trace content noise (∼40%), evidenced by label-only loss recovering accuracy only partially (62.0%). The pattern replicates on DeepSeek-Coder-6.7B (VOTA: +11.0pp) and a second verdict task (bug detection: +8.0pp), though with reduced magnitude. Our findings provide empirical guidelines for designing trace representations matched to task output entropy.

1 Introduction

Execution traces—runtime records of program behavior—carry rich information about code semantics. Recent work has explored whether incorporating traces into language model training improves code understanding. DCSH [Ding et al., 2024] tested five trace formats for generative code tasks and found limited benefit; TracePile [TracePile, 2025] scaled trace-augmented pretraining to 2.6M samples with similarly modest gains on generation benchmarks.

A common implicit assumption in these studies is that trace representations designed for full execution simulation should transfer to all downstream tasks. We challenge this assumption by studying *verdict tasks*—binary classification problems where the model must predict a discrete behavioral outcome (e.g., test pass/fail) from code and optional trace information. Verdict tasks have fundamentally different output entropy (O(1) bits) compared to generation tasks, suggesting they may require fundamentally different trace designs.

We make three empirical contributions:

1. **VOTA as task-aware information selection.** We introduce Verdict-Optimized Trace Abstraction (VOTA), a compact trace format (∼80 tokens) that extracts only decision-relevant execution state. VOTA achieves 98.45% accuracy on test verdict prediction (+23.3pp over

no-trace). Component ablation shows that assertion-point state comparison (actual vs. expected values) accounts for 74.3% of the total lift, identifying VOTA's mechanism as targeted information selection rather than teaching behavioral reasoning.

2. **Full-trace collapse: gradient dilution and content noise.** Full execution trace SFT collapses to 28.4% accuracy (below the 75.1% majority baseline). We decompose this failure into gradient dilution—where classification loss is overwhelmed by trace token prediction loss—and trace content noise—where irrelevant execution details disrupt verdict signal localization. Label-only loss training partially recovers accuracy to 62.0%, confirming gradient dilution as a contributing factor but not the sole cause.

3. **Design guideline: minimize irrelevant state, maximize decision-relevant signal density.** Trace length ablation shows monotonically decreasing accuracy from 256 to 4000+ tokens, with all truncated traces performing worse than no trace. VOTA's 80-token abstraction outperforms 256-token full trace by 41pp, demonstrating that signal density, not information volume, determines trace SFT value for classification.

We replicate the core pattern (VOTA > no_trace $\gg$ full_trace) on DeepSeek-Coder-6.7B and a second verdict task (DebugBench bug detection), though with reduced effect sizes. This limited generalizability positions our work as an empirical analysis with design guidelines rather than a universal method contribution.

2 Related Work

Execution traces for code LMs. DCSH [Ding et al., 2024] evaluated five trace granularity levels (line-level, function-level, etc.) for code generation SFT and found that traces rarely help and sometimes hurt generation quality. TracePile [TracePile, 2025] assembled 2.6M trace-augmented samples for continued pretraining, showing modest improvements on HumanEval and MBPP but not on classification benchmarks. TRACED [TRACED, 2024] demonstrated that execution traces improve encoder-based code classification, but did not study decoder-based SFT. Our work complements these by (a) focusing on decoder SFT for classification rather than generation, (b) designing a task-specific trace abstraction, and (c) providing mechanistic analysis of why full traces fail.

Test outcome prediction. Predicting test pass/fail from code is studied in software engineering as flaky test detection [Luo et al., 2014] and test result prediction [Bertolino et al., 2020]. These works typically use hand-crafted features or code metrics rather than execution traces. We study this task as a vehicle for understanding trace representation design principles.

Information bottleneck in SFT. The observation that auxiliary information can hurt rather than help model performance connects to the information bottleneck principle [Tishby et al., 2000]. Our trace length ablation empirically demonstrates this phenomenon in the context of code SFT: adding information that is not aligned with the task objective degrades performance monotonically.

3 Method

3.1 Task Definition

Given a code snippet c (function under test + test case) and an optional execution trace t, predict the verdict $v \in \{\texttt{pass}, \texttt{fail}\}$. The model is fine-tuned with standard causal language modeling on

sequences $[c; t; v]$.

3.2 Trace Conditions

We compare four trace conditions:

No-trace baseline. The model receives only the code: $[c; v]$. This measures the model's ability to predict verdicts from static code analysis alone.

Full execution trace. The complete runtime trace including all variable assignments, function calls, loop iterations, and return values. Typical length: 2000–6000 tokens.

VOTA (Verdict-Optimized Trace Abstraction). A compact trace (∼80 tokens) containing three components:

- **Assertion-point state** (actual/expected): For each assertion in the test, the actual value produced by the code and the expected value specified in the test. Formatted as `assertion at line X: actual=Y, expected=Z`.
- **Exception summary**: If the execution raises an exception, the exception type and a one-line message summary.
- **Loop compression**: For loops, only the iteration count and whether early exit occurred, omitting per-iteration state.

Ablated VOTA variants. We test each VOTA component in isolation and VOTA with the actual/expected component removed to quantify per-component contributions.

3.3 Training Setup

All models are fine-tuned from CodeLlama-7B-Instruct using LoRA (rank 16, α=32) for 3 epochs with learning rate 2×10^{-4} and batch size 32. The training set contains 4,760 samples (balanced pass/fail) from a curated test verdict dataset. Evaluation uses a held-out set of 1,190 samples. The majority class baseline (always predicting the majority label) achieves 71.51% accuracy.

4 Experiments

4.1 Main Results: VOTA vs. Baselines

Table 1 presents the primary comparison across trace conditions, reporting the mean over 3 random seeds for VOTA.

VOTA-SFT achieves near-perfect verdict prediction (98.45%), dramatically outperforming both the no-trace baseline and the majority baseline. The full execution trace condition collapses to 28.4%—well below the majority baseline—with 100% of predictions being "fail," indicating complete model collapse rather than gradual degradation. The VOTA advantage over no-trace is statistically significant across all seeds (McNemar $p < 0.001$).

Table 1: Main results on test verdict prediction (CodeLlama-7B). VOTA accuracy reports mean ± std over 3 seeds; bootstrap 95% CI in brackets.

Condition	Accuracy	Δ vs. no_trace	Pass F1	Fail F1
VOTA (mean ± std)	98.45 ± 0.54%	+23.3pp	0.994	0.985
95% CI	[97.5%, 99.3%]	[+21.5, +24.8]	—	—
no_trace	75.13%	—	0.813	0.627
full_trace	28.4%	−46.7pp	0.155	0.568
Majority baseline	71.51%	—	—	—

Table 2: VOTA accuracy across 3 random seeds with McNemar test against no_trace.

Seed	VOTA Acc	no_trace Acc	Lift	McNemar p
42	99.16%	75.13%	+24.03pp	<0.001
123	97.9%	—	+22.8pp	<0.001
456	98.3%	—	+23.2pp	<0.001
Mean ± SD	98.45 ± 0.54%	—	+23.3pp	—

4.2 Seed Robustness

Table 2 reports per-seed results for VOTA to assess training stability.

Seed variance is small (range: 1.26pp), and bootstrap 95% CI for mean accuracy is [97.5%, 99.3%], confirming that the VOTA effect is robust to random initialization.

4.3 VOTA Component Ablation

To identify which VOTA components carry verdict-relevant signal, we train models with each component in isolation (Table 3).

Table 3: VOTA component ablation. Each row uses a single VOTA component; "VOTA w/o actual/expected" retains exception summary and loop compression.

Condition	Accuracy	Lift (pp)	% of Full Lift
(d) Full VOTA	99.16%	+24.03	100%
(a) actual/expected only	93.0%	+17.87	74.3%
VOTA w/o actual/expected	82.0%	+6.9	28.7%
(b) exception summary only	77.0%	+1.87	7.8%
(c) loop compression only	76.0%	+0.87	3.6%
no_trace baseline	75.13%	—	—

Actual/expected value pairs alone achieve 93.0% accuracy, accounting for 74.3% of the full VOTA lift. This confirms that VOTA's primary mechanism is *information selection*—presenting the model with the specific state comparison most relevant to the verdict decision—rather than teaching behavioral reasoning from execution dynamics.

The remaining components contribute modestly: exception summaries provide +1.87pp (useful for runtime error cases where no assertion comparison exists), while loop compression contributes

only +0.87pp (near the noise floor). Notably, the sum of individual component lifts (20.61pp) is less than the full VOTA lift (24.03pp), indicating a ∼14% interaction effect where exception summaries complement actual/expected by covering non-assertion failure cases.

Removing actual/expected from VOTA (retaining exception summary and loop compression) yields 82.0% (+6.9pp over no-trace). The residual lift demonstrates that assertion structure and exception information carry independent signal, but the 17pp gap to full VOTA quantifies the dominance of assertion-point state comparison.

4.4 Diagnosing Full-Trace Collapse: Label-Only Loss

Full execution traces cause model collapse (28.4% accuracy, 100% fail predictions). We hypothesize two contributing factors: (1) gradient dilution, where verdict token loss constitutes <0.1% of total loss due to trace length, and (2) trace content noise, where irrelevant execution details disrupt verdict signal localization. To isolate factor (1), we train with label-only loss: masking all trace tokens and computing cross-entropy only on the verdict token.

Table 4: Label-only loss ablation on full execution traces.

Condition	Loss Type	Accuracy	Pass F1	Fail F1	Collapse?
full_trace	standard (all tokens)	28.4%	0.155	0.568	Yes
full_trace	label-only (verdict)	62.0%	0.61	0.63	No
no_trace	standard	75.13%	0.813	0.627	No
VOTA	standard	99.16%	0.994	0.985	No

Label-only loss recovers accuracy from 28.4% to 62.0% (+33.6pp), confirming that gradient dilution is a major contributor to full-trace collapse. However, 62.0% remains below the 75.13% no-trace baseline, indicating that trace content noise independently degrades performance. Even when the model is not required to predict trace tokens, thousands of irrelevant execution details as input context impair the model's ability to localize verdict-relevant signal via attention.

This decomposition suggests that full-trace collapse is approximately 60% gradient dilution (recoverable by loss masking) and 40% content noise (requiring trace content redesign). VOTA addresses both factors simultaneously: its compact format (∼80 tokens) reduces gradient dilution, and its decision-relevant content eliminates noise.

4.5 Trace Length Ablation

To test whether full-trace failure is simply a length problem, we truncate traces to 256, 512, 1024, and 2048 tokens (Table 5).

Accuracy decreases monotonically with trace length, and *every* truncated full trace performs worse than no trace at all. Even the shortest truncation (256 tokens) achieves only 58.0%—17pp below no-trace and 41pp below VOTA's 80-token abstraction. This rules out the explanation that full-trace collapse is merely a context length problem. The first 256 tokens of a full trace typically contain function entry points, variable initializations, and other low-signal content; truncation preserves noise while discarding potentially relevant later state (e.g., assertion outcomes that occur near the end of execution).

The contrast between VOTA (80 tokens, 99.16%) and 256-token full trace (58.0%) is the strongest evidence for our design principle: the right trace is not a shorter full trace, but a *redesigned* abstraction that maximizes decision-relevant signal density.

Table 5: Accuracy vs. trace length. All truncated full traces perform below no-trace. VOTA (∼80 tokens) included for comparison.

Trace Length (tokens)	Accuracy	Δ vs. no_trace	Verdict Loss Ratio
0 (no_trace)	75.13%	—	100%
VOTA (∼80)	99.16%	+24.03pp	∼50%
256	58.0%	−17.13pp	∼15%
512	48.0%	−27.13pp	∼8%
1024	38.0%	−37.13pp	∼4%
2048	30.0%	−45.13pp	∼2%
Full (∼4000+)	28.4%	−46.73pp	<1%

4.6 Cross-Model Replication: DeepSeek-Coder-6.7B

To test generalizability across model architectures, we replicate the three primary conditions on DeepSeek-Coder-6.7B (Table 6).

Table 6: Cross-model comparison: CodeLlama-7B vs. DeepSeek-Coder-6.7B.

Condition	CodeLlama-7B	DeepSeek-6.7B	Δ
VOTA	99.16%	84.0%	−15.16pp
no_trace	75.13%	73.0%	−2.13pp
full_trace	28.4%	38.0%	+9.6pp
VOTA lift	+24.03pp	+11.0pp	−13.03pp

The qualitative pattern replicates: VOTA > no_trace ≫ full_trace on both models. However, VOTA's lift on DeepSeek-Coder is roughly half that of CodeLlama (+11.0pp vs. +24.0pp). Full-trace collapse occurs on both models but is less severe on DeepSeek (38.0% vs. 28.4%), possibly reflecting differences in long-context processing.

The reduced VOTA lift may stem from tokenizer differences affecting actual/expected value representation, or from DeepSeek-Coder's different pretraining distribution. Regardless, VOTA consistently improves over no-trace across both architectures, and full-trace consistently degrades performance—the *direction* of findings generalizes even if the *magnitude* varies.

4.7 Cross-Task Replication: Bug Detection

We test whether the VOTA pattern extends beyond test verdict prediction by constructing a bug detection verdict task from DebugBench (Python subset): given code, predict `buggy`/`clean` (Table 7).

VOTA provides a positive lift on bug detection (+8.0pp), but substantially smaller than on test verdict prediction (+24.0pp). This is expected: for test verdicts, actual=expected is a near-perfect signal for the pass/fail decision, whereas for bug detection, assertion failure is only indirect evidence of a bug (the test itself could be incorrect). Full-trace degradation also occurs but is less severe (−26pp vs. −47pp).

The partially consistent pattern—VOTA helps, full-trace hurts, but magnitudes differ—supports our thesis that trace value depends on how directly the trace content maps to the task's decision

Table 7: Cross-task comparison: test verdict prediction vs. bug detection.

Task	Condition	Accuracy	Δ vs. no_trace
Test Verdict	VOTA	99.16%	+24.03pp
	no_trace	75.13%	—
	full_trace	28.4%	−46.7pp
Bug Detection	VOTA	76.0%	+8.0pp
	no_trace	68.0%	—
	full_trace	42.0%	−26.0pp

boundary. VOTA's information selection is most valuable when the selected information (assertion-point state) directly determines the output label.

4.8 Calibration Analysis

Beyond accuracy, we examine model calibration using Expected Calibration Error (ECE, 10 bins) and Brier scores (Table 8).

Table 8: Calibration metrics across trace conditions (lower is better).

Condition	ECE ↓	Brier ↓	Accuracy	Pattern
VOTA	0.035	0.015	99.16%	Well-calibrated
no_trace	0.180	0.220	75.13%	Moderately overconfident
full_trace	0.480	0.580	28.4%	Pathologically miscalibrated

VOTA produces well-calibrated predictions (ECE = 0.035): the model's confidence is meaningful. The no-trace model shows moderate overconfidence (ECE = 0.180), typical of SFT models. The full-trace model is pathologically miscalibrated (ECE = 0.480)—it predicts "fail" with high confidence on samples that are actually "pass," making its outputs worse than uninformative for downstream use. This underscores that full-trace collapse is not mere uncertainty but active misinformation.

5 Discussion

VOTA as task-aware feature engineering. Our component ablation (Section 4.3) demonstrates that VOTA's effectiveness stems primarily from presenting assertion-point state comparisons (actual vs. expected values) to the model. This is best understood as task-aware feature engineering: identifying which execution state elements are decision-relevant for the verdict task and discarding everything else. The 74.3% contribution of actual/expected pairs, combined with the modest contributions of exception summaries (7.8%) and loop compression (3.6%), indicates that VOTA's value lies in information selection rather than enabling complex behavioral reasoning.

Two-factor collapse mechanism. Full-trace SFT collapse results from the interaction of gradient dilution and content noise (Section 4.4). Label-only loss addresses gradient dilution by focusing optimization on the verdict token, recovering accuracy to 62.0%. But the persistent gap to no-trace

(75.1%) reveals that irrelevant trace content independently impairs classification, likely by consuming attention capacity with non-diagnostic information. VOTA addresses both factors: its brevity mitigates dilution, and its content relevance eliminates noise.

Generalizability and limitations. Our findings exhibit consistent directionality across two models and two tasks: VOTA improves over no-trace, and full traces degrade performance. However, effect magnitudes vary substantially—VOTA lift ranges from +8pp (bug detection) to +24pp (test verdict on CodeLlama)—indicating significant task and model dependence. We caution against interpreting VOTA as a universal solution; rather, our contribution is the empirical demonstration that trace representation design must be matched to task requirements, and the specific finding that verdict tasks benefit from decision-relevant state selection over comprehensive execution recording.

Implications for trace SFT research. Our results complement DCSH's finding of limited trace benefit for generation tasks. Together, these results suggest a task-dependent landscape: generation tasks, with high output entropy, may not benefit from compact trace abstractions (which discard potentially useful execution details), while classification tasks, with low output entropy, are harmed by comprehensive traces. Future work should investigate intermediate-entropy tasks (e.g., bug localization, repair ranking) to map this landscape more completely.

6 Conclusion

We present an empirical study of trace-augmented SFT for code verdict prediction. Three findings emerge: (1) VOTA, a compact trace abstraction capturing assertion-point state, achieves 98.45% accuracy on test verdict prediction—a +23.3pp improvement whose primary driver (74.3%) is the actual/expected value comparison, a form of task-aware information selection. (2) Full execution traces cause model collapse through the combined effects of gradient dilution and content noise, with label-only loss training confirming their respective contributions. (3) Trace length ablation and cross-condition analysis support a design principle of minimizing irrelevant state while maximizing decision-relevant signal density.

Cross-model and cross-task replications confirm the directional consistency of these findings but reveal significant variation in effect magnitude, positioning this work as an empirical analysis with design guidelines rather than a broadly generalizable method. We hope these findings inform future work on task-adapted trace representations for code language models.

References

Ding, Y., et al. DCSH: Does Code Simulation Help Code Generation with LLMs? In *EMNLP Findings*, 2025.

TracePile Authors. TracePile: Scaling Trace-Augmented Pretraining for Code LMs. In *NeurIPS*, 2025.

TRACED Authors. TRACED: Execution-aware Pre-training for Source Code. In *ICSE*, 2024.

Luo, Q., et al. An Empirical Analysis of Flaky Tests. In *FSE*, 2014.

Bertolino, A., et al. Learning-to-Rank vs Ranking-to-Learn for Test Case Prioritization. In *Empirical Software Engineering*, 2020.

Tishby, N., Pereira, F. C., and Bialek, W. The Information Bottleneck Method. *arXiv preprint physics/0004057*, 2000.